

Ereditarieta

Ereditarieta

Come creare classi specializzate partendo da classi piu generali.

Cosa significa ereditarieta

L'ereditarieta permette di creare una classe partendo da un'altra.

La classe piu generale contiene dati e metodi comuni. La classe piu specifica li riusa e aggiunge cio che le serve.

Esempio:

- `Persona` e generale
- `Studente` e una persona con una matricola

extends

In Java si usa `extends` .

```
public class Persona {  
    String nome;  
  
    void saluta() {  
        System.out.println("Ciao, sono " + nome);  
    }  
}
```

```
public class Studente extends Persona {  
    String matricola;  
}
```

`Studente` eredita il campo `nome` e il metodo `saluta` .

Usare la classe derivata

```
public class Esempio {  
    public static void main(String[] args) {  
        Studente studente = new Studente();  
  
        studente.nome = "Luca";  
        studente.matricola = "A123";  
  
        studente.saluta();  
        System.out.println(studente.matricola);  
    }  
}
```

Output:

```
Ciao, sono Luca  
A123
```

Classe base e classe derivata

La classe da cui parti viene spesso chiamata:

- classe base
- superclasse
- classe padre

La classe che eredita viene chiamata:

- classe derivata
- sottoclasse
- classe figlia

Sono parole diverse per descrivere la stessa relazione.

Sovrascrivere un metodo

Una sottoclasse può ridefinire un metodo ereditato.

```
public class Studente extends Persona {
    String matricola;

    @Override
    void saluta() {
        System.out.println("Ciao, sono lo studente " + nome);
    }
}
```

`@Override` dice a Java e a chi legge: "sto sovrascrivendo un metodo della classe base".

Costruttori e super

Se la classe base ha un costruttore con parametri, la sottoclasse deve chiamarlo con `super`.

```
public class Persona {
    private String nome;

    public Persona(String nome) {
        this.nome = nome;
    }

    public String getNome() {
        return nome;
    }
}
```

```
public class Studente extends Persona {
    private String matricola;

    public Studente(String nome, String matricola) {
        super(nome);
        this.matricola = matricola;
    }
}
```

`super(nome)` chiama il costruttore di `Persona`.

Quando usarla con attenzione

L'ereditarietà è utile quando una classe è davvero un tipo più specifico di un'altra.

`Studente` è una `Persona` : ha senso.

Ma non usarla solo per "prendere codice". A volte è meglio mettere un oggetto dentro un altro invece di ereditarlo.

Regola pratica:

se puoi dire "X è un tipo di Y", l'ereditarietà potrebbe avere senso.

Se devi dire "X usa Y" o "X contiene Y", probabilmente serve un'altra struttura.